Semi-Empirical Dynamic Stall Models

Jeet Adsule (AE19B025), Kirtan Patel (AE19B038),
Muthukumar PR (AE19B042)

Abstract

This report examines the use of semi-empirical models in the study of dynamic stall, a phenomenon that occurs when a lifting surface experiences unsteady aerodynamic forces. Semi-empirical models are introduced as a potential solution to limitations of current computational models. These models incorporate empirical data and physical principles to predict the onset and evolution of dynamic stall. The report explores two semi-empirical models : the ONERA Model and the Snel Model.

The strengths and weaknesses of each model are analyzed, and their predictions were tried to be compared with experimental data, which was attempted to be obtained using openFOAM. The report concludes that semi-empirical models offer a promising approach to predicting dynamic stall, but more research is needed to fully validate their accuracy and applicability to different flow conditions and airfoil geometries.

1 Introduction

1.1 Dynamic Stall

Dynamic stall is a phenomenon that occurs when an airfoil experiences a rapid change in angle of attack, typically at high speeds. When an airfoil approaches or exceeds its critical angle of attack, the flow over the airfoil becomes unsteady, leading to separation of the flow and the formation of vortices. These vortices can cause a rapid fluctuation in the lift and drag forces on the airfoil, which can result in large variations in the aircraft's motion or control surfaces.

Dynamic stall can occur in a variety of situations, such as during takeoff and landing of aircraft, rotorcraft, and wind turbines. It can also occur in other applications involving fluid dynamics, such as in propellers and hydrofoils.

Dynamic stall can have both positive and negative effects on the performance and stability of the airfoil or aircraft. On one hand, dynamic stall can provide additional lift and increase the maximum lift coefficient of the airfoil, which can be beneficial for takeoff and landing. On the other hand, dynamic stall can also result in a loss of lift and control authority, which can be detrimental to the aircraft's stability and safety.

Understanding the causes and effects of dynamic stall is important for the design and operation of aircraft, rotorcraft, and other applications involving fluid dynamics. Various models and simulation methods have been developed to study and predict dynamic stall under different conditions.

1.2 Dynamic Stall in Biomimicry

In nature, many animals are able to take advantage of dynamic stall to enhance their flight or swimming performance.

Biomimicry is the practice of studying and emulating nature's solutions to design and engineering challenges. In the field of aerodynamics, researchers have looked to nature for inspiration in designing more efficient and maneuverable aircraft. One area of interest is how animals utilize dynamic stall to their advantage.

For example, birds such as the albatross and the peregrine falcon are able to perform high-speed dives and turns by using dynamic stall to generate lift and maneuverability. Similarly, some fish are able to rapidly change direction and accelerate by using their fins to create vortices and take advantage of dynamic stall.

Researchers have attempted to replicate these natural mechanisms in aircraft and underwater vehicles. One approach is to design wing or fin shapes that can generate vortices and take advantage of dynamic stall. Another approach is to develop control systems that can manipulate the flow over the surface to induce dynamic stall and enhance maneuverability.

One example of biomimicry inspired by dynamic stall is the design of flexible wings that can change shape and adapt to different conditions, similar to the wings of birds. By using flexible materials and mechanisms to control wing shape, researchers have been able to reduce drag and improve lift in certain flight conditions.

Another example of biomimicry inspired by dynamic stall is the design of rotor blades for wind turbines. Researchers have looked to the wings of birds and bats to develop new rotor blade designs that can operate more efficiently in turbulent or dynamic wind conditions. By incorporating flexible materials and adapting the blade shape to the changing flow conditions, these new designs have shown promise in improving the efficiency and stability of wind turbines.

1.3 Empirical and Semi-Empirical Methods

The empirical method and semi-empirical method are two approaches used in scientific research to develop models or theories based on experimental observations.

The empirical method involves the collection of data through observation or experimentation and the formulation of theories or laws based on those observations. The data is then used to test and refine the theories or laws. The empirical method is often used in fields such as physics, chemistry, and biology.

On the other hand, the semi-empirical method involves combining empirical data with theoretical models or assumptions to develop a more complete understanding of a phenomenon. This method involves using mathematical or computational models that are based on empirical data but also incorporate theoretical concepts or assumptions. The semi-empirical method is often used in fields such as materials science and condensed matter physics.

The empirical method relies solely on experimental data and may not provide a complete understanding of a phenomenon. The semi-empirical method, on the other hand, may rely on theoretical assumptions that may not accurately reflect the real world. However, the semi-empirical method can often provide a more complete and accurate understanding of a phenomenon by combining both empirical data and theoretical models.

2 ONERA Model

The ONERA model for dynamic stall was first developed by researchers (McAlister, Lambert, and Petott) from ONERA, the French National Aerospace Research Centre in 1984. They sought to predict the unsteady loads on an airfoil that is experiencing dynamic stall. In this paper, they describe the mathematical model used to predict the dynamic load and then compare it with experimental observations for the same parameters to validate the model. The model utilizes characteristics of two differential equations to express the aerodynamic responses as a function of time.

2.1 Equations

The ONERA model takes inputs of angle of attack (as a function of time) along with a set of coefficients which empirically depend on the airfoil which is used. The model also assumes (perhaps incorrectly) that the values for the instantaneous force and moment loads do not vary significantly from their steady flow counterparts.

The dynamic load, after experimental observations, was modeled as the sum of two independent loads: one whose effect is prominent below static stall angle and could be represented by a first order differential equation and the second which was needed to represent oscillations above static stall (given by a second order differential equation).

$$F = F_1 + F_2 \tag{1}$$

$$\dot{F}_1 + \lambda F_1 = \lambda F_1 + (\lambda s + \sigma)\dot{\alpha} + s\ddot{\alpha} \tag{2}$$

$$\ddot{F}_2 + \alpha \dot{F}_2 + r F_2 = -(r\Delta + e\dot{\Delta}) \tag{3}$$

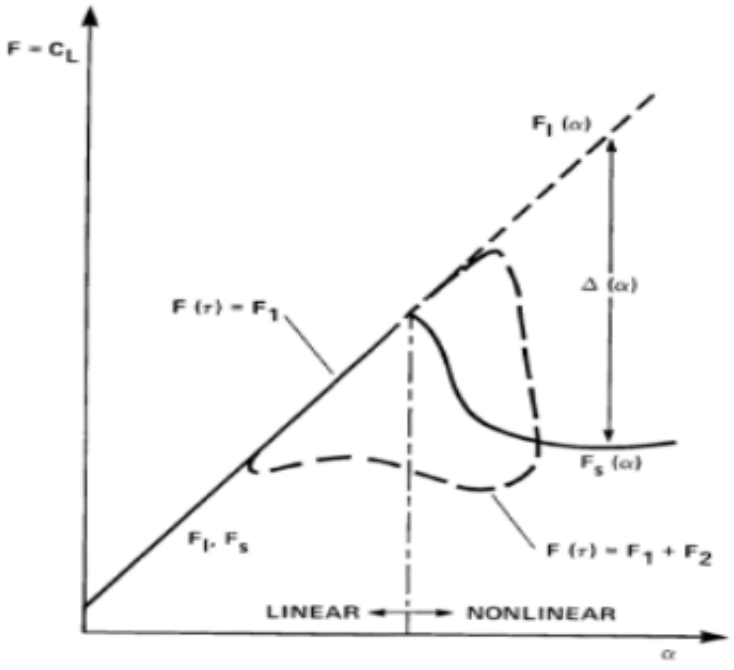


Figure 1: Plotting the differential relations [1]

The functions F_1 and Δ are both properties of the airfoil used and can be derived from the lift polar curve. F_1 (later non-dimensionalized and represented as $C_{L,pot}$ or $C_{L,p}$) is defined here as the interpolation of the linear portion of the lift curve with constant slope (as shown in figure). Δ (later non-dimensionalized and represented as ΔC_L or ΔC_L^{pot}) is then defined as the difference between F_1 and the actual steady lift (F_s) for that specific angle of attack.

ΔC_L is given as

$$\Delta C_L = c_{l,pot} - c_{l,stat} \tag{4}$$

or

$$\Delta C_L^{pot} = \frac{dC_L^{st}}{d\alpha}|_{\alpha_0} \sin(\alpha - \alpha_0) - C_L^{st} \tag{5}$$

Both these definitions are plotted in the figures left and right and it is observed that they are similar for the range of angles that are used.

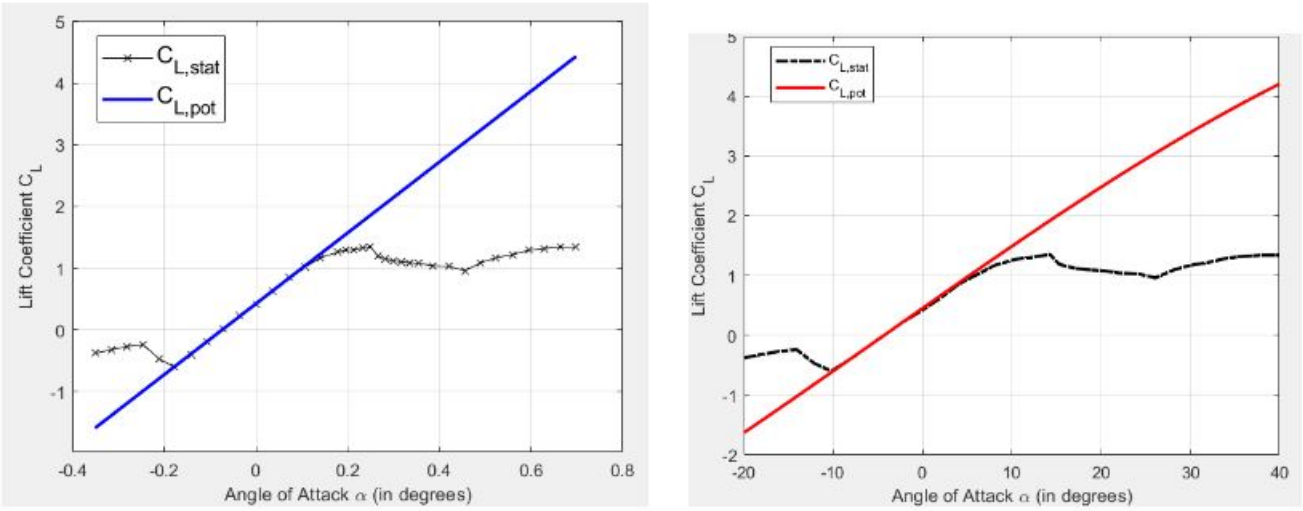


Figure 2: C_L^{pot} obtained using different definitions

Rewriting the original equations non-dimensionally,

$$\dot{c_{l,1}} + \lambda_L c_{l,1} = \lambda_L c_{l,pot} + (\lambda_L s_L + \sigma_L)\dot{\alpha} + s_L \ddot{\alpha} \tag{6}$$

The coefficients λ, σ, and s are functions of angle of attack but since they tend to vary little with α, they are taken to be constant for a specific airfoil. Their exact values are determined by experiments but can be replaced by flat plate values in case experimental data is unavailable

$$\ddot{c_{l,2}} + a_L \dot{c_{l,2}} + r_L c_{l,2} = -(r_L \Delta c_l + e_L \dot{\Delta c_l}) \tag{7}$$

$$r_L = [r_0 + r_2(\Delta c_l)^2]^2 \tag{8}$$

$$a_L = a_0 + a_2(\Delta c_l)^2 \tag{9}$$

$$e_L = e_2(\Delta c_l)^2 \tag{10}$$

The coefficients in the second equation, r, e and a depend on ΔC_L and another set of coefficients r_0, r_2, a_0, a_2 and e_2 which are also obtained from experimental data but in their absence the values for a mean airfoil shape can be substituted.

It also has to be mentioned that all the derivatives used in the above equations are with respect to non-dimensionalized time 's'. Time (t) is non-dimensionalized by using the relation $s = (2U/c)/t$, where U is the free stream velocity and c is the airfoil's chord length.

2.2 Numerical Solution

Now that we know how the model is formulated, the equations were coded into MATLAB. The appropriate constants used in the references are substituted so as to validate the results obtained. A free stream velocity of 10 m/s along with a chord length of 1.46 m for the NACA 4415 airfoil are used as a Reynolds number of the order of 10^6 is used for most of the reference plots. ($Re = Uc/\nu$ and $\nu = 1.46e-5\,m^2/s$ is used)

ONERA

Parameter	Value
λ_L	0.17
s_L	π
σ_L	2π
a_0	0.3
a_2	0.2
e_2	−2.86
r_0	0.2
r_2	0.2

Figure 3: Coefficients used in the paper [**Holierhoek2012**] to solve the differential equations

2.3 Code Explanation

Since the available lift curve data [2] for the airfoil used (NACA 4415) is discrete, an interpolation function is used to interpolate the curve for any angle in the given domain. Following that, the $C_{L,pot}$ and ΔC_L are calculated from the interpolated steady lift coefficient curve using either definition mentioned. The user is prompted to enter parameters of the input angle of attack function: the reduced frequency k, mean angle of attack α_M and amplitude α_P.

$$\alpha(t) = \alpha_M + \alpha_P \sin(\omega t) = \alpha_M + \alpha_P \sin(ks) \tag{11}$$

A time interval of two complete oscillations is taken as the domain and appropriate time steps are made. Angle of attack (and its derivatives), $C_{l,p}$, ΔC_L (and its derivative) and coefficients r_L, e_L and a_L for each of these time steps are calculated. These parameters are then passed into the equation, given necessary boundary conditions (at $t = 0$, $C_{L,1}$ is the same as $C_{L,S}$,$C_{L,2} = 0$ and $dC_{L,2}/dt = 0$) which is solved by the in-built 'dsolve' function for each time instance. The results are then plotted.

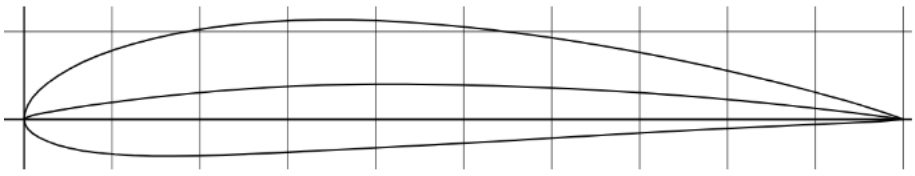


Figure 4: NACA 4415 airfoil which is used for all the solutions

2.4 Plots

The results from the MATLAB code solved for the dynamic lift coefficient (obtained as a function of time) is plotted against angle of attack. The graph has a hysteresis loop, as in, it has multiple values of lift coefficient for a value of α. This is because as the airfoil completes one complete oscillation it passes through one angle twice (once when it increases and once when it decreases).

Curves are plotted for a certain k and α_M,α_P as a base reference and then following that, the angle is slightly increases for the second plot keeping k the same and the third plot with higher k but for the same angles. This is done to observe the individual effects of the frequency and mean/amplitude. The second oscillation (as mentioned in the code explanation) is used to plot the curves as the first oscillation has some erratic variations which may be because the solution hasn't converged yet.

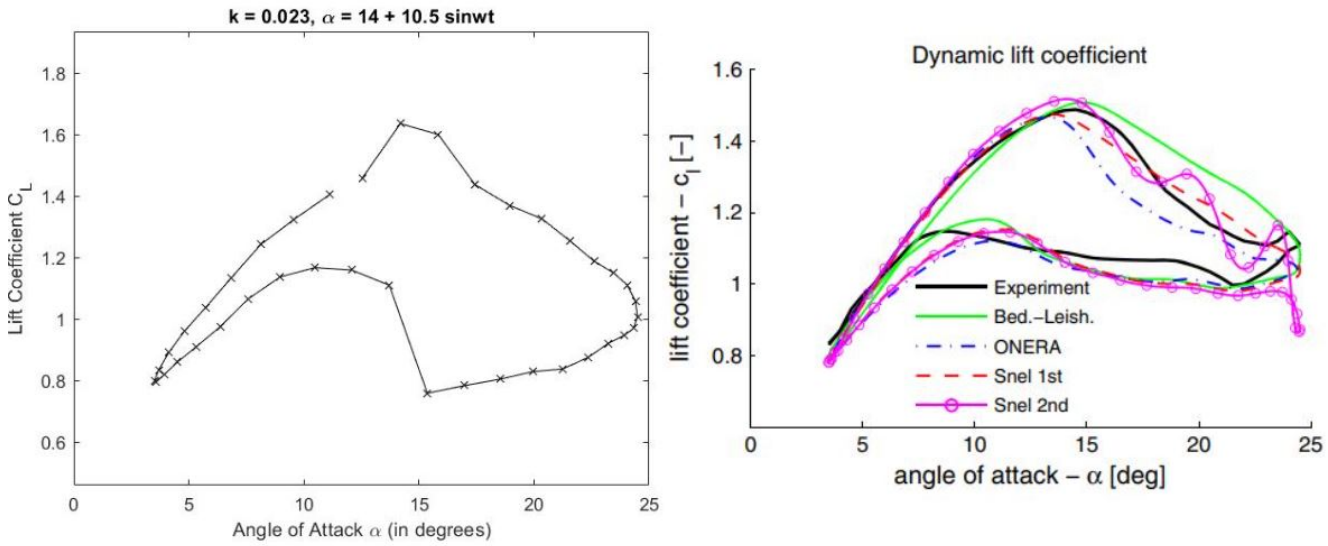


Figure 5

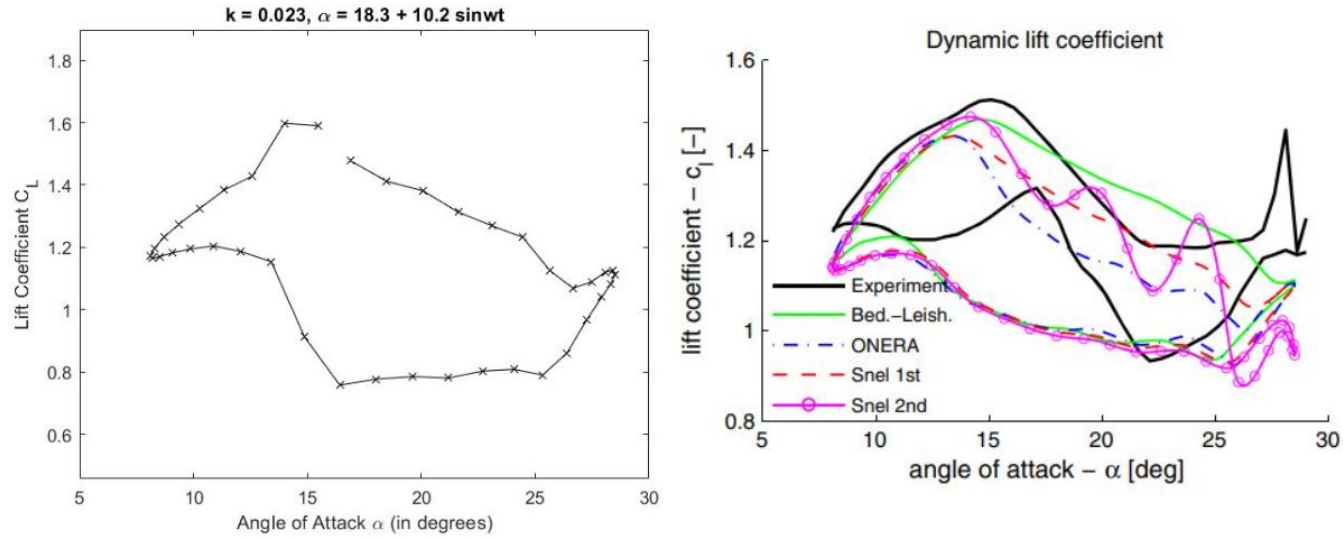


Figure 6

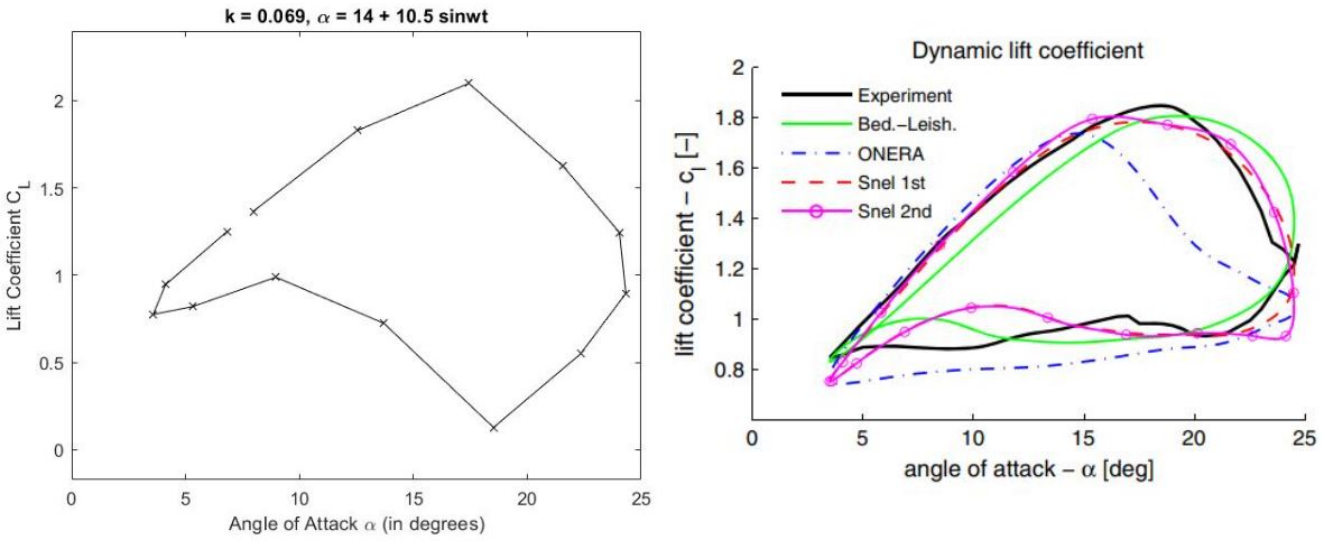


Figure 7

The results match the reference [3] graphs fairly well for lower angles but has some variations for higher α values. This disparity is most likely due to the higher impact of the ΔC_L term in that range of angle of attack.

2.5 Advantages and Drawbacks

- The main advantage of using the ONERA model is that the set of differential equations can be linearized and solved easily.
- The model loses its accuracy as both the amplitude of oscillation and the reduced frequency increase.
- This model uses a number of empirical constants which have to be predetermined experimentally for the model to produce accurate results, which may not always be possible.

3 Snel Model

This model was developed by H. Snel in 1997. It is different from the ONERA model in that it models the dynamic lift as a set of corrections over the static lift. These two correction terms are given by their first order linear and second order differential equations respectively.

$$C_L = C_L^{st} + \Delta C_{L,1} + \Delta C_{L,2} \tag{12}$$

The first term $\Delta C_{L,1}$ models the effect of the forcing frequency response and is described by a first order ordinary differential equation similar to a 'damper spring model'.

$$\Delta \dot{C}_{L,1} + T_u^{-1} c_{10} \Delta C_{L,1} = \Delta \dot{C}_L^{pot} \tag{13}$$

where corrections to the older model model have included a slightly modified version of this coefficient which depends on the sign of $\dot{\alpha} C_L^{pot}$ and is given by

$$c_{10} = \begin{cases} \frac{1+0.5\Delta C_L^{pot}}{8+640 T_u |\dot{\alpha}|} & \text{if } \dot{\alpha} C_L^{pot} > 0 \\ \frac{1+0.5\Delta C_L^{pot}}{8-480 T_u |\dot{\alpha}|} & \text{if } \dot{\alpha} C_L^{pot} \leq 0 \end{cases} \tag{14}$$

and $T_u = c/2U$

The second term $\Delta C_{L,2}$ describes higher frequency dynamics and is represented as a second order differential equation.

$$T_u^2 \Delta \ddot{C}_{L,2} + c_{21} \Delta \dot{C}_{L,2} + c_{20} \Delta C_{L,2} = 0.015(-0.15 \Delta C_L^{pot} + 0.05 \Delta \dot{C}_L^{pot}) \tag{15}$$

where

$$c_{20} = k_s^2 (1 + 3\Delta C_{L,2}^2)(1 + 3\dot{\alpha}^2) \tag{16}$$

$$c_{21} = \begin{cases} 60 T_u k_s (-0.01(\Delta C_L^{pot} - 0.5) + 2\Delta C_{L,2}^2) & \text{if } \dot{\alpha} C_L^{pot} > 0 \\ 2 T_u k_s & \text{if } \dot{\alpha} C_L^{pot} \leq 0 \end{cases} \tag{17}$$

where $k_s = 0.2$, Strouhal Vortex Shedding Frequency

It is important to note that in the case that the inequality is greater than zero, the second order differential equation becomes non-linear as a time derivative of the state variable $\Delta C_{L,2}$ is multiplied by its square. The constant k_s is referred to as the Strouhal vortex shedding frequency which is taken to have a value of 0.2. Some papers comparing Snel's model use the terms 'Snel 1st' and 'Snel 2nd' to indicate using only the first correction term or using both terms respectively.

The MATLAB code for solving the Snel model numerically is very similar to the code used for the ONERA model. However, the 'ode45' solver has been used in this case instead of the 'dsolve' as it has to deal with non-linear differential equations and is much faster.

3.1 Plots

The simulation has been used to recreate results published in previous publications [4]. The model's results have been classified into three categories: light stall, moderate stall and deep stall based on their angle of attack input relatively.

3.1.1 Light Stall

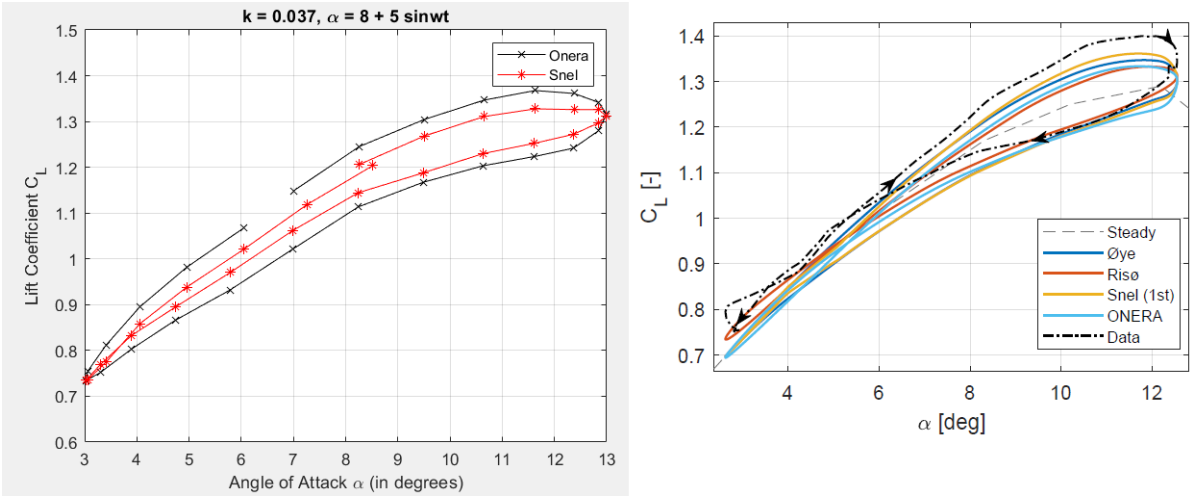
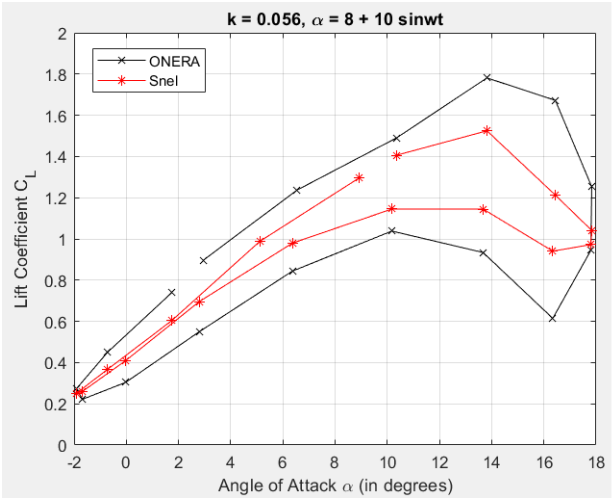


Figure 8

3.1.2 Moderate Stall



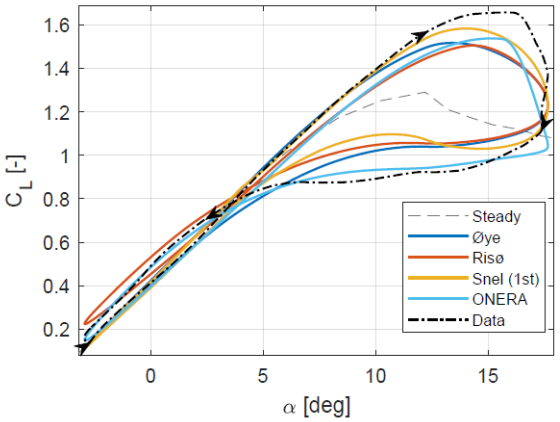
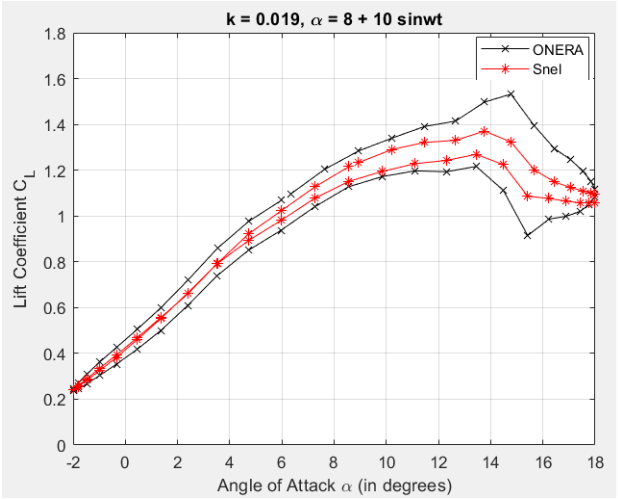


Figure 9



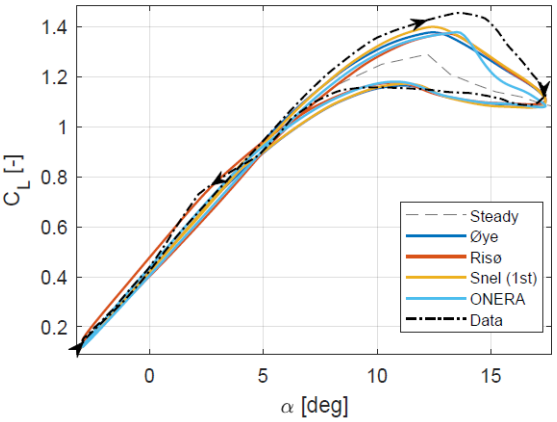
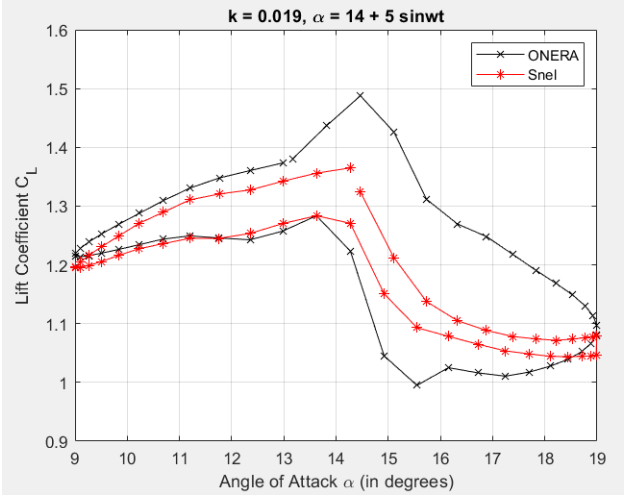


Figure 10

3.1.3 Deep Stall



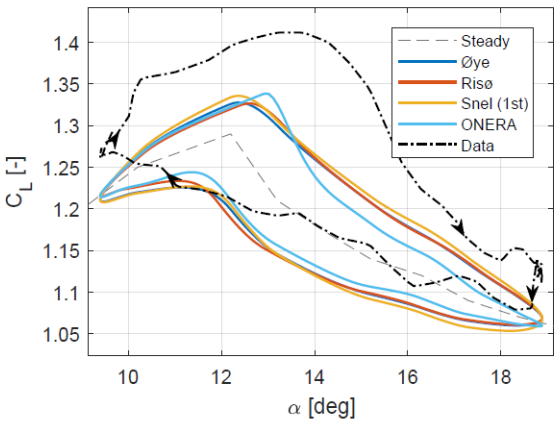
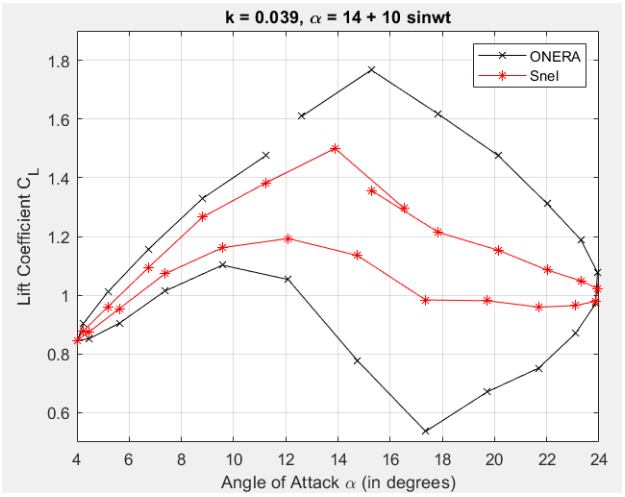


Figure 11



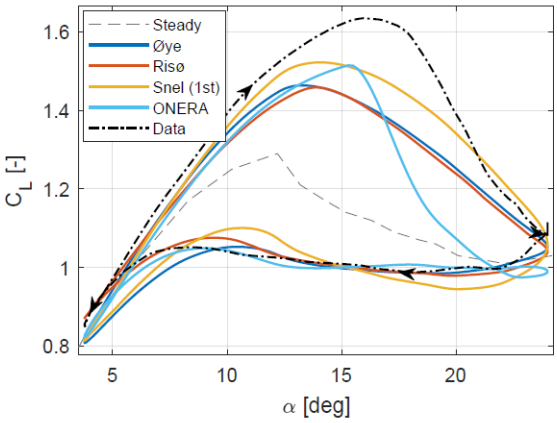
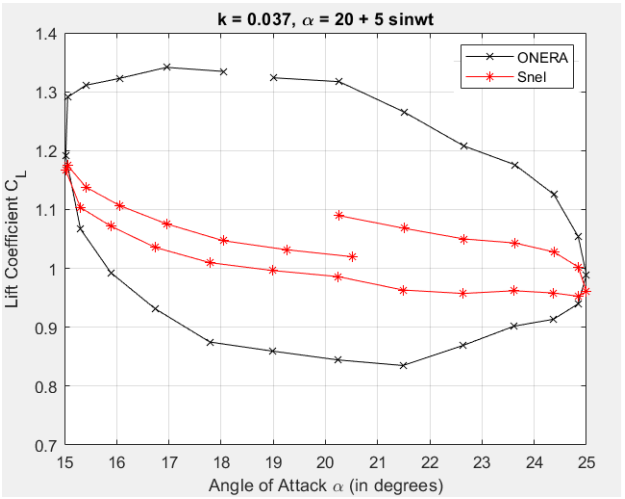


Figure 12



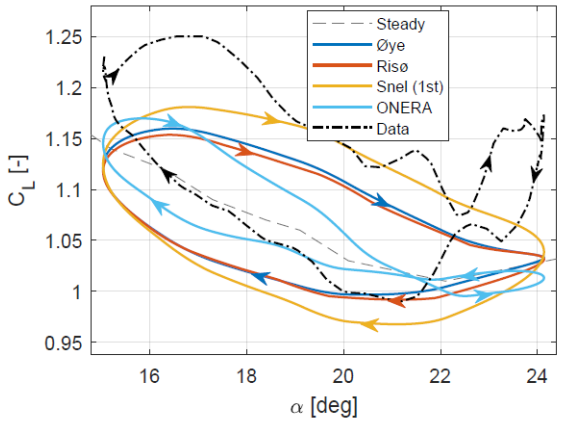


Figure 13

3.2 Advantages and Drawbacks

- It produces accurate results comparable to other models even without using airfoil-specific parameters.
- Slightly harder to solve numerically than ONERA model because some of its coefficients contain the state variable making it non-linear in $\Delta C_{L,2}$

4 CFD Analysis using openFOAM

4.1 Process flow

An attempt to obtain the experimental results was made using CFD simulation using openFOAM.

The utilization of OpenFOAM for simulation involves a combination of both proprietary and open-source resources. Specifically, when it comes to pre-processing tasks such as constructing watertight geometry and surface mesh in STL format, it is more convenient to work outside the open-source environment. The process flow, which doesn't involve interfacing with other programs for structural simulations, is outlined below.

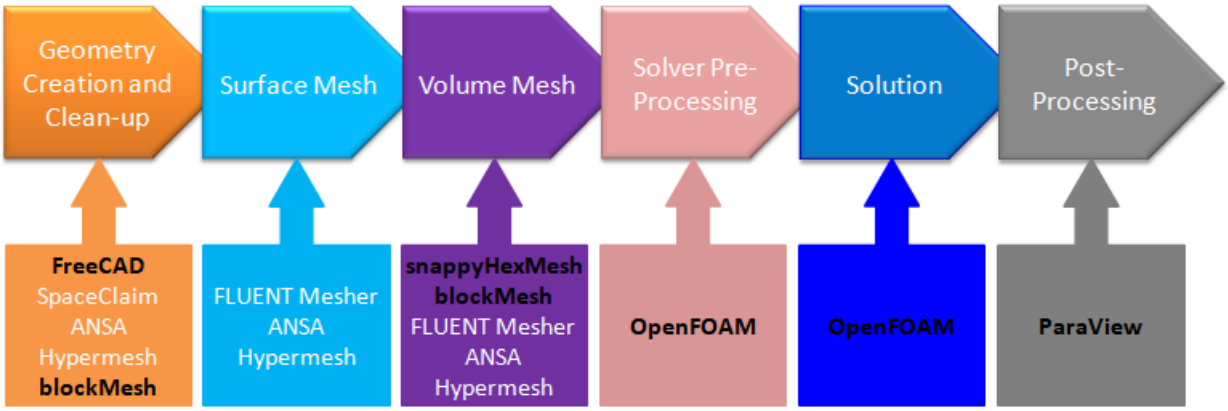


Figure 14: OpenFOAM Workflow [5]

The computational fluid dynamics (CFD) process flow using OpenFOAM typically involves several steps which can be catergorized as follows, starting from pre-processing to post-processing. These steps are outlined below:

1. **Pre-processing :** This step involves creating a mesh and defining the boundary conditions for the simulation. The user can generate the mesh using OpenFOAM's built-in mesh generation tools or import it from other software. Boundary conditions such as inlet velocity, pressure, and wall conditions are defined at this stage.

2. **Running the simulation :** After the pre-processing is completed, the user can start the simulation using OpenFOAM's solver. The solver is selected based on the type of simulation and the physics of the problem being solved. The simulation can be run in parallel using multiple cores or distributed over a cluster.

3. **Post-processing :** Once the simulation is complete, the results can be visualized and analyzed using OpenFOAM's post-processing tools. These tools include software for data visualization, contour plots, streamlines, and 3D visualization. Users can also export simulation data to other software for further analysis.

4. **Iteration :** Based on the results of the simulation, the user may need to make changes to the model or boundary conditions and re-run the simulation. This iterative process allows users to refine the model and achieve more accurate results.

The above steps provide a general overview of the CFD process flow using OpenFOAM. However, the specific steps and tools used may vary depending on the nature of the problem being solved. OpenFOAM provides a wide range of solvers and tools that can be customized and extended to meet the needs of the user.

4.2 openFOAM Simulation Setup

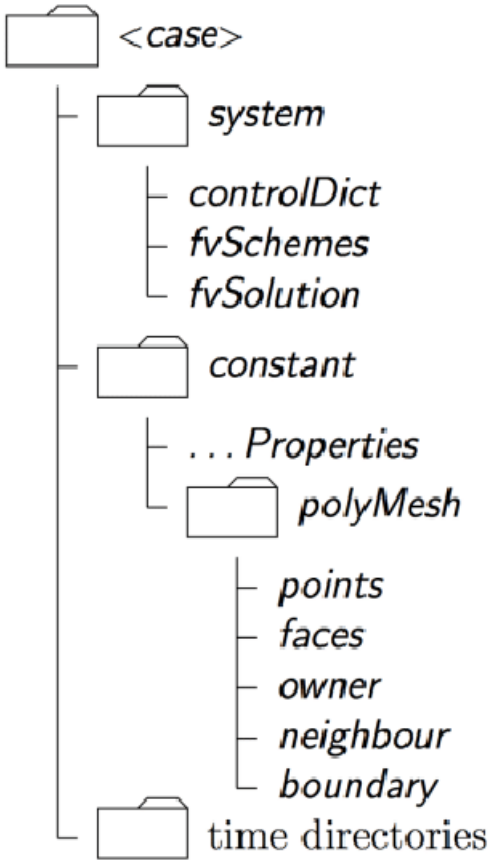


Figure 15: OpenFOAM Directory

A brief description of the core elements of the setup directory is given below :

- **system** : contains numerical solver
  - **controlDict** : contains all the parameters controling the simulation which includes time marching process, time step. Used rho pimpleFOAM is the transient solver used.
  - **fvSchemes** : is the file which governs the discretization schemes used for each and every parameter individually i.e velocity or temperature can have different discretization schemes. To keep things simple, Gauss Limited Linear scheme has been used for all the parameters.
  - **fvSolution** : this file governs the sovler parameters over discretization i.e. which kind of solver is used for every parameter along with their tolerances.
- **constant** : directory contains information about various constants used in the process like, the thermal and fluid properties and the geometry used.
  - **polyMesh** : is a subdirectory containing defined parts of geometry such as boundary and mesh points
- **time directories** : it contains initial conditions as well as the results of different time outputs returned from the simulation. Since our simulation didn't run, this directory only contains initial values.

References

[1] D. Petot K. W. McAlister O. Lambert. *Application of the ONERA Model of Dynamic Stall*. 1984.

[2] G.M. Gregorek M. J. Hoffmann R. Reuss Ramsay. *Effects of Grit Roughness and Pitch Oscillations on the NACA 4415 Airfoil*. 1996.

[3] et al. J. G. Holierhoek2012. *Comparing Different Dynamic Stall Models*. 2012. DOI: 10.1002/we.548.

[4] Mark Faber. *A Comparison of Dynamic Stall Models and their Effect on Instabilities*. 2018.

[5] *CFD Simulations with openFOAM*. URL: http://www.cfdyna.com/Home/OpenFOAM.html (visited on 05/01/2023).

A MATLAB Simulation Codes

A.1 ONERA Model

```
clear;
%% Calculate Clp and delCl for NACA 4415
data = readmatrix('4415_1m.csv');
aoa = data(:,2)*pi/180;
cls = data(:,3);
coef = polyfit(aoa(6:14), cls(6:14), 1);
clp = aoa*coef(1)+coef(2);

plot(aoa,cls,'k','Linewidth',2);
hold on;
plot(aoa,clp,'b');
hold off;
grid on;

%% Constants from the ONERA model
lamL = 0.17;
sigL = 2*pi;
sL = pi;
a0 = 0.3;
a2 = 0.2;
e2 = -2.86;
r0 = 0.2;
r2 = 0.2;
Re = 1e6;
V = 10;
nu = 1.46e-5;
c = Re*nu/V;
k = input("Enter reduced frequency (k): ");
w = k*V/c;
%% Alpha as function of time
ts = 1;
t = 0:ts:(2*pi/w)*2;
s = (2*V/c)*t;
ss= ts*(2*V/c);
am = input("Enter mean angle of attack \alpha_m: ");
ap = input("Enter amplitude of angle of attack \alpha_a: ");
a = (am + ap*sin(w*t))*pi/180;
cls_ = spline(aoa,cls,a);
clp_ = a*coef(1)+coef(2);
delcl = clp_ - cls_;
deldotcl = gradient(delcl(:)) ./ gradient(s(:));
%{
plot(aoa,cls,'k','Linewidth',2);
hold on;
plot(a,cls_,'-o','MarkerSize',5);
%}
%% Dependants
```

```
rL = (r0 + r2*delcl.^2).^2;
aL = a0 + a2*delcl.^2;
eL = e2*delcl.^2;
%% Differential Equation 1
adot = gradient(a(:)) ./ gradient(t(:))*0.5*(c/V);
addot = gradient(adot(:)) ./ gradient(t(:))*0.25*(c/V)^2;
syms cl1(s_);
t_ = 0;
cl1_ = zeros(length(s),1);
while t_ < length(t)
    t_ = t_+1;
    ode = diff(cl1,s_) + lamL*cl1 == lamL*clp_(t_) + (lamL*sL +
       sigL)*adot(t_) + sL * addot(t_);
    cond = cl1(0) ==  cls(interp1(aoa*180/pi,1:length(aoa),am,'
       nearest'));
    csol = dsolve(ode,cond);
    cl1_(t_) = subs(csol,s_,s(t_));
end
%figure
%plot(a,cl1_,'k');
%% Differential Equation 2
syms cl2(s_);
t_ = 0;
cl2_ = zeros(length(s),1);
while t_ < length(t)
    t_ = t_+1;
    cond1 = cl2(0) == 0;
    cond2 = cl2(20) == 0;
    ode = diff(diff(cl2,s_),s_)+aL(t_)*diff(cl2,s_)+rL(t_)*cl2 ==
       ...
        -rL(t_)*delcl(t_) - eL(t_)*deldotcl(t_);
    conds = [cond1,cond2];
    dsol = dsolve(ode,conds);
    cl2_(t_) = subs(dsol,s_,s(t_));
end
sol = cl1_((2*pi/w):(2*pi/w)*2)+cl2_((2*pi/w):(2*pi/w)*2);
figure
plot(a((2*pi/w):(2*pi/w)*2)*180/pi,sol,'k-x');
hold on
grid on
legend('ONERA');
ylim([min(sol)-0.1 max(sol)+0.1]);
xlabel("Angle of Attack \alpha (in degrees)");
ylabel("Lift Coefficient C_L");
title("k = "+k+", \alpha = "+am+" + "+ap+" sinwt");
savefig('onera.fig');

%{
cl1 = zeros(length(t),1);
cl1(1) = 1.25;
```

```
t_ = 1; % time index, should be multiplied with time step to get t
while t_ < length(t)
    t_ = t_+1;
    cl1(t_) = (lamL*clp_(t_-1)*ss + (lamL*sL+sigL)*adot(t_-1)*ss
       ...
        + sL*(adot(t_)-adot(t_-1))+cl1(t_-1)*(1-lamL*ss));
end
figure
plot(a,cl1,'k');

t_ = 2;
cl2 = zeros(length(t),1);
cl2(1) = 0;
cl2(2) = 0;
while t_ < length(t)
    cl2(t_+1) = cl2(t_)*(0.5*(c/V)^2 + 0.5*aL(t_)*ts*(c/V) - rL(t_
       )*ts^2)...
        + cl2(t_-1)*0.25*(c/V)^2 - rL(t_)*delcl(t_)*ts^2 ...
        - eL(t_)*ts*(delcl(t_+1)-delcl(t_))*0.5*(c/V);
    cl2(t_+1) = cl2(t_+1)/(0.25*(c/V)^2+0.5*aL(t_)*(c/V)*ts);
    t_ = t_+1;

end
figure
plot(a,cl2+cl1,'k','Linewidth',2);
%}
```

A.2 Snel Model

```
1
2  clear;
3  %% Calculate Clp and delCl for NACA 4415
4  data = readmatrix('4415_1m.csv');
5  global aoa cls Tu ks a adot deldotclpot
6  aoa = data(:,2)*pi/180;
7  cls = data(:,3);
8  aser = (-20:0.1:40)*pi/180;
9  %plot(aser*180/pi,clstat(aser,cls),'k');
10 %hold on;
11 res = clstat(aser,cls);
12 clalsl = gradient(res(:)) ./ gradient(aser(:));
13 [ ~, ix ] = min( abs( res-0 ) );
14 global zls azl
15 zls = clalsl(ix); % zero lift slope
16 azl = aser(ix); % zero lift aoa
17 Re = 1e6;
18 V = 10;
19 nu = 1.46e-5;
20 c = Re*nu/V;
21 k = input("Enter reduced frequency (k): ");
22 w = k*V/c;
23 ts = 1;
24 t = 1:ts:(2*pi/w)*2+1;
25 s = (2*V/c)*t;
26 Tu = c/(2*V);
27 am = input("Enter mean angle of attack \alpha_m: ");
28 ap = input("Enter amplitude of angle of attack \alpha_a: ");
29 a = (am + ap*sin(w*t))*pi/180;
30 %{
31 figure
32 plot(aser*180/pi,clstat(aser,cls),'k-.','Linewidth',2);
33 legend('C_{L,stat}');
34 hold on
35 grid on
36 plot(aser*180/pi,delclpot(aser)+clstat(aser,cls),'r','Linewidth
     ',2);
37 legend('C_{L,pot}');
38 xlabel("Angle of Attack \alpha (in degrees)");
39 ylabel("Lift Coefficient C_L");
40 %}
41 %% cl1
42
43 d = delclpot(a);
44 deldotclpot = gradient(d(:)) ./ gradient(t(:));
45 adot = gradient(a(:))./gradient(t(:));
46 %plot(a*180/pi,deldotclpot,'k');
47 %figure
48 %plot(t,a*180/pi,'k');
```

```
cl1_ = zeros(length(t),1);
for ti = t
    cl10 = 0;
    if (adot(ti)*delclpot(a(ti))) > 0
        c10 = (1+0.5*delclpot(a(ti)))/(8+640*0.5*(c/V)*abs(adot(ti
          )));
    else
        c10 = (1+0.5*delclpot(a(ti)))/(8-480*0.5*(c/V)*abs(adot(ti
          )));
    end
    [tv,cl1] = ode45(@(tv,cl1) deldotclpot(ti) - 2*(V/c)*cl1 *c10,
       t, cl10);
    cl1_(ti) = interp1(tv,cl1,ti);
end
temp = clstat(a,cls);
temp2 = reshape(cl1_(t),1,[]);
%plot(a*180/pi,temp+temp2,'k')

%% cl2
syms tv cl2
cl2_ = zeros(length(t),1);
for ti = t
    ic = [0,0];
    %c20 = ks^2*(1+3*cl2(1)^2)*(1+3*adot(ti));
    ks = 0.2;
    if (adot(ti)*delclpot(a(ti))) > 0
        [tv,cl2] = ode45(@(tv,cl2) delcl2plus(tv,cl2,ti),t,ic) ;
    else
        [tv,cl2] = ode45(@(tv,cl2) delcl2minus(tv,cl2,ti),t,ic) ;
    end
    cl2_(ti) = interp1(tv,cl2(:,1),ti);
end
temp3 = reshape(cl2_(t),1,[]);
tempn = temp(end-round(2*pi/w):end);
figure
plot(a(end-round(2*pi/w):end)*180/pi,temp(end-round(2*pi/w):end)+
   temp2(end-round(2*pi/w):end)+temp3(end-round(2*pi/w):end),'r-*'
   )
hold on
plot(a(end-round(2*pi/w):end)*180/pi,temp(end-round(2*pi/w):end)+
   temp2(end-round(2*pi/w):end),'g-*')

ylim([0.7 1.4]);
legend('Snel');
xlabel("Angle of Attack \alpha (in degrees)");
ylabel("Lift Coefficient C_L");
title("k = "+k+", \alpha = "+am+" + "+ap+" sinwt");
savefig('snel.fig')
function res = clstat(a,cser)
global aoa
```

```
res = interp1(aoa,cser,a);
end
function res = delclpot(a)
global zls azl cls
res = zls*sin(a-azl) + -clstat(a,cls);
end
function res = delcl2plus(tv,cl2,ti)
global Tu ks a adot deldotclpot
res1 = cl2(2);
res2 = -cl2(2)*60*Tu*ks*(-0.01*(delclpot(a(ti))-0.5)+2*cl2(1)^2)
   ...
            -ks^2*(1+3*cl2(1)^2)*(1+3*adot(ti))*cl2(1) + 0.015
               *(-0.15*delclpot(a(ti))+0.05*deldotclpot(ti));
res = [res1;res2];
end
function res = delcl2minus(tv,cl2,ti)
global Tu ks a adot deldotclpot
res1 = cl2(2);
res2 = -cl2(2)*2*Tu*ks...
            -ks^2*(1+3*cl2(1)^2)*(1+3*adot(ti))*cl2(1) + 0.015
               *(-0.15*delclpot(a(ti))+0.05*deldotclpot(ti));
res = [res1;res2];
end
```

B openFOAM Simulation Files

B.1 controlDict

```
/*--------------------------------*- C++ -*----------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     | Website:  https://openfoam.org
    \\  /    A nd           | Version:  9
     \\/     M anipulation  |
\*---------------------------------------------------------------------------*/
FoamFile
{
    format      ascii;
    class       dictionary;
    object      controlDict;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

application     rhoPimpleFoam;

startFrom       latestTime;

startTime       0;

stopAt          endTime;

endTime         100;

deltaT          1e-3;

writeControl    adjustableRunTime;

writeInterval   0.01;

purgeWrite      0;

writeFormat     ascii;

writePrecision   8;

writeCompression off;

timeFormat      general;

timePrecision   6;

runTimeModifiable true;

functions
{
```

```
48      #includeFunc MachNo
49      #includeFunc residuals
50  }
51
52  // ********************** //
```

B.2 fvConstraints

```
1   /*--------------------------------*- C++ -*----------------------------------*\
2     =========                 |
3     \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
4      \\    /   O peration     | Website:  https://openfoam.org
5       \\  /    A nd           | Version:  9
6        \\/     M anipulation  |
7   \*---------------------------------------------------------------------------*/
8   FoamFile
9   {
10      format      ascii;
11      class       dictionary;
12      object      fvConstraints;
13  }
14  // * * * * * ** * * * * * * * * * * * * * * * * * * //
15
16  limitp
17  {
18      type        limitPressure;
19
20      minFactor   0.1;
21      maxFactor   2;
22  }
23
24  limitT
25  {
26      type        limitTemperature;
27
28      min         101;
29      max         1000;
30
31      selectionMode all;
32  }
33
34  //************************************************* //
```